

The dictionary picture: an intuition-level companion to the rank-1 leakage model

June 2026

This note restates the working model of behavior leakage (`rank1_leakage_model.pdf`, same folder) at the level of intuition, with no derivations. The parent note has the assumptions, relaxations, predictions, and experiment status; this one has the cartoon that makes them readable.

1 The picture in five claims

1. **Pretraining learns a dictionary of behaviors.** The model acquires a collection of behaviors b — sycophancy, marker emission, emergent misalignment — each carried by a direction in the residual stream at some layer. The direction plays two roles that should be named separately: *adding* a steering vector d_b elicits the behavior; *projecting* the state onto a read-out direction r_b measures it. In the idealized case one direction plays both roles ($d_b = r_b$); they can come apart, with a steering vector nearly orthogonal to the read-out that downstream circuitry converts (the router case).
2. **The state under a context is a weighted sum over the dictionary.** A context c is summarized by a vector v_c read while the model processes it. (Most generally the context representation is some function of the *KV cache* — that is all the response computation ever sees of c — and the pooled v_c is the cheapest member of that family; whatever the gate later misses traces back to what the pooling threw away.) The model’s own context-to-state map f then puts the activations somewhere, and that somewhere decomposes approximately over the dictionary:

$$f(c) \approx \sum_b \alpha_b(c) d_b + \text{residual}.$$

The point of the weights: the *associations* $c \rightarrow b$ that the model learned in pretraining ARE the coefficients $\alpha_b(c)$. Every context expresses many behaviors at once (its bundle); “the assistant is not a villain” is a statement about weights, not about the dictionary.

3. **Post-training edits the weight functions, never the dictionary.** Fine-tuning adds no behavior to the collection. In the lazy regime a small weight update cannot build new states or circuitry; it can only re-bind contexts to states that already exist — which is why a behavior is implantable only insofar as it is *elicitable* (prompting is how you locate the destination state that training will bind to). Shaping assistant behavior, marker implants, and EM induction are all edits of $\alpha_b(\cdot)$, differing in which weights move and for which contexts.
4. **Output is read out linearly, then squashed.** The state is not passed directly through a softmax. The chain is

state $f(c) \rightarrow$ logits $z_t = \langle W_U[t], f(c) \rangle$ per token $t \rightarrow$ softmax over the vocabulary $\rightarrow$ output,

so the softmax is over *tokens*, not over behaviors. For the marker, behavior expression and a single logit coincide (z_{marker}), which is what makes it the clean testbed. For sycophancy, expression is a sequence-level property scored by a judge; every token still exits through the softmax, but the behavior is not one softmax entry. Because the softmax normalizes, behaviors compete at emission (marker vs end-of-turn) and log-probabilities compress near the ceiling — the reason logits are stored alongside log-probabilities.

5. **Two typed functions carry everything.** Keep them apart:

- $f : c \mapsto f(c)$ — one state per context, no behavior argument. *This is what training edits.*
- $A[b, c] = \langle r_b, f(c) \rangle$ — a scalar association table, rows indexed by behaviors, columns by contexts. *This is what experiments measure.*

The behavior vector v_b is neither: it is the state f reaches when α_b dominates — where the model *is* when it is performing b .

2 The whole model is one rank-one nudge to the table

Training on (c', b) drags the source state $v_{b'} = f(c')$ toward the behavior state v_b , stopping at an achieved state $v_{b''}$ set by the dose (steps $\times$ lr); the write $w = v_{b''} - v_{b'}$ is therefore *measured, not assumed*. The minimal weight edit that implements this applies the same write to every context, scaled by a base-model similarity, the gate $g(c) = \langle v_{c'}, v_c \rangle / \|v_{c'}\|^2$. In table coordinates the entire parent note compresses to:

$$\Delta A[b, c] = \underbrace{\langle r_b, w \rangle}_{\text{behavior factor}} \cdot \underbrace{g(c)}_{\text{context factor}}$$

— a **rank-one update of the association table itself**. You intended to edit one cell (b, c') . The minimal edit cannot touch one cell; it smears along both axes:

- *down the columns* (across contexts): every context moves in proportion to its gate — that smear is **leakage** (P2);
- *across the rows* (across behaviors): every behavior moves in proportion to its slope $\langle r_b, w \rangle$ — that smear is **cross-behavior transfer** (P6).

Leakage and cross-behavior transfer are the same event seen along the two axes of the table. This also gives leakage a clean definition in dictionary coordinates: leakage *is* off-target table movement — coefficients $\alpha_b(c)$ changing for behavior–context pairs the fine-tune never targeted. The intended edit is one cell; everything else that moves is generalization nobody asked for.

3 Cast of characters

Object	Type	Role	Moved by training?
v_c	vector	context key (input side), read while processing c	no (base-model object)
f	map	contexts $\rightarrow$ states; holds the associations	yes — the only thing edited
$f(c)$	state	where the model sits under c	moves when f moves
$\alpha_b(c)$	scalar	association strength of $c \rightarrow b$ (the weights)	moves (this <i>is</i> the edit)
d_b	direction	steering: add to elicit b	no (dictionary)
r_b	direction	read-out: project to measure b	no (and directly observable)
v_b	state	where the model is when performing b (target)	no (dose-independent)
$v_{b'}, v_{b''}$	states	source state before / achieved after training	$v_{b''}$ is the outcome
$w = v_{b''} - v_{b'}$	vector	the write: <i>what</i> leaks	created by training
$g(c)$	scalar	the gate: <i>who</i> leaks	no (computed from base v_c)

4 The best-effort question

A way to pose the program’s target without committing to any particular gate: drop all restrictions. Out of every possible embedding of contexts — nonlinear, any function of the KV cache — pick the one most discriminatory for the behavior at hand: the best detector the base model’s activations can support of “should this context express b ?”. Suppose fine-tuning could gate the behavior on that best embedding. Is there still leakage?

- **If leakage is inevitable even at best effort, the reason has to be named.** The candidate reason: lazy edits have limited expressivity. A small weight update can only gate on directions the base model already represents (approximately) linearly; if separating the source from the bystanders requires a nonlinear function of the KV cache that the base model never linearized, no small edit can read it. Leakage would then be forced by the editing mechanism rather than by information — the contexts are distinguishable in principle, just not by anything a rank-limited edit can see. On this view, inevitable leakage = the gap between the best discriminator and the best discriminator expressible as a linear gate on frozen features.
- **If leakage is evitable, the answer should be constructive: exhibit the edit.** The contrastive multi-pair edit is the linear-case candidate: solving the kernel system subtracts the common mode of near-parallel context keys (cosine ≈ 0.92 across persona prompts) and zeroes the gate on the enumerated negatives. Its known limitation is exactly the right test: protection extends only to (roughly) the span of the negatives trained against, so leakage to contexts *off* the panel measures how far best effort actually reaches.

Assuming linear embeddings sharpens the question into geometry. A rank-one edit’s gate is $g(c) = \langle k, v_c \rangle$ for some effective key k ; zero leakage to every bystander while still firing on the source requires k orthogonal to all bystander keys but not to v_c — possible exactly when the source key has a usable component outside the bystanders’ span. So in the linear world: leakage is evitable iff the source context is linearly separable from the bystander distribution in key space; the optimal key is the discriminant direction; and contrastive negatives are an estimator of that direction. What stays open is the continuum problem: “all contexts” is not an enumerable panel, and whether a discriminant fitted against K negatives generalizes to the rest of context space is an empirical question, not a theorem.

5 Where the cartoon bends

- **The weighted sum is the idealized case.** Dictionary directions are not orthogonal and do not span the state space (superposition); the decomposition is literal for the marker at read-out (one unembedding row), an empirical finding for EM (a convergent single direction), and an open bet elsewhere.
- **Some behaviors are fetched, not resident.** In the router case the write recruits attention circuitry that pulls the behavior from the context’s own token values; the gate is then attention-shaped and near-binary rather than a smooth similarity.
- **“At some layer” is behavior-specific.** The marker reads best late (layers 19–24), sycophancy early (1–8); there is no universal dictionary layer.
- **Whether two implants add is open.** The sum-of-writes picture across jointly trained behaviors is a prediction (P4, the task-arithmetic analogue), not an assumption, and is so far untested in a diagnostic regime.
- **Everything assumes the lazy regime.** If training moves the dictionary itself (the rich regime), contexts’ keys, the directions, and every base-model predictor go stale together.

For the derivation of the rank-one edit (assumptions A1–A4 and their relaxations), the prediction list P0–P8, and per-experiment status, see `rank1_leakage_model.pdf`.